

Semantic Search RAG vs. *Moment RAG*

Two retrieval pipelines over one YouTube transcript — a flat semantic-search baseline, and a moment-aware system that decomposes, retrieves hybrid, re-ranks, and cites the exact second.

The video chosen — and *why* it fits.

A short, dense explainer is a clean knowledge source: many distinct ideas packed into a few minutes, each a natural "moment."

THE VIDEO

“How does a Vector Database work?”

A ~10-minute explainer that walks an employee-handbook search example from SQL to embeddings to vector search.

- Topic-dense — embeddings, similarity, indexing, thresholds, chunk overlap — each a distinct moment
- Clean auto-captions → 312 timestamped cues, no manual transcription
- On-topic: the system explains the very technology it runs on



▶ Vector Databases

312 cues · ~2,400 tokens

youtube.com/watch?v=VVNYQKDL5s

Understanding the *Moment RAG* codebase.

The reference puts intelligence on the query side and resolves retrieval to timestamped moments, not just documents.

Enrich *once* at ingestion; think *hard* at query time.

Ingestion (once)

Transcripts are timed cues; chunks carry *start/end timestamps* — that is what makes a chunk a citeable moment.

Each chunk is pre-tagged with the *hypothetical questions* it answers (HyDE-at-ingestion) and embedded as an extra retrieval branch, alongside dense and BM25 sparse vectors.

Query time

Self-query infers filters → *decompose* into sub-queries → hybrid retrieve (dense + BM25 + question vectors), *RRF-fused*.

A *cross-encoder re-ranks* the shortlist, then chunk hits roll up to moments — each citation deep-links into the video.

Baseline: *flat* semantic search.

Fixed chunks, one embedding lookup, stuff-and-answer — the conventional RAG most teams ship first.

Chunk → embed → index → *answer*

01

Chunk

Fixed ~256-token windows,
25% overlap → 13 chunks.

→

02

Embed

OpenAI text-embedding-3-
small → a 13 × 1536 matrix.

→

03

Index

In-memory NumPy cosine
index — or ChromaDB (HNSW).

→

04

Answer

Top-k by cosine, stuffed into
gpt-4o-mini.

Moment RAG: think on the *query* side.

Semantic moments with timestamps, question-indexing, hybrid retrieval, re-ranking, and cited answers.

Intelligence moves to the *query* side

01

Segment & Enrich

Semantic breakpoints → 25 timestamped moments; each tagged with the questions it answers (99 Q-vectors).



02

Decompose

One question → 2–4 focused sub-queries.



03

Hybrid + Rerank

Dense + questions + BM25, RRF-fused, then cross-encoder re-rank.



04

Cite

Moments roll up to a gpt-4o-mini answer with &t=timestamp deep-links.

“

On “how does a vector database find similar vectors?” the baseline returned “the context does not provide an answer.” Moment RAG answered it — with six timestamped citations.

SAME TRANSCRIPT • SAME MODELS
THE DIFFERENCE IS RETRIEVAL

One short video, measured both ways

25

semantic moments vs. 13 fixed chunks from the same transcript

```
segment_moments()
```

99

hypothetical-question vectors added as a HyDE retrieval branch

```
enrich_moment()
```

~2¢

total OpenAI cost for a full top-to-bottom run

```
embeddings + gpt-4o-mini
```

KEY FINDINGS

Why moment-aware retrieval wins

Same source, same models — the gains come entirely from how retrieval is structured.

- Decomposition covers multi-part questions the baseline answers only halfway
- Question-indexing (HyDE) matches intent even when wording differs from the transcript
- Hybrid + RRF balances meaning (dense) against exact terms (BM25)
- Cross-encoder re-rank pushes the truly on-point moment to the top
- Timestamped citations make every claim verifiable in one click

What it *doesn't* do yet — and where it goes next.

Limitations

Higher cost & latency: one LLM enrichment call per moment, plus decomposition and re-rank on every query.

The NumPy index is exact but brute-force — fine for one video, not for millions of vectors.

Segmentation and answers inherit caption quality and the short explainer's depth.

Improvements

Cache enrichment and persist vectors (ChromaDB / Qdrant) so ingestion runs only once.

Add self-query metadata filters and scale to a multi-video corpus.

Tune segmentation, top-k, and rerank depth; add an LLM-judge eval to measure quality.

MODULE 3 · AGENTIC RAG

Better retrieval, *not* a bigger model.

Calvin Nguyen ·

github.com/calvnguyen/semantic_search_moment_rag